

ASSESSMENT TOOL

CO3-AT2-CASE STUDY

Data Analytics Platform: Merge Sort vs Quick Sort

Submitted by

Name: Bhuvanesh. S

Registration No: 192521004

Q5. Case Study: Data Analytics Platform (Merge vs Quick Sort) A data analytics platform must process large datasets efficiently for reporting. Analyze the use of Merge Sort and Quick Sort in this scenario. Identify key issues such as memory usage, speed, and stability. Apply divide and conquer concepts to compare both algorithms. Analyze their performance under different conditions. Design a solution selecting the best algorithm and justify your choice

1. Description of the Real-World Problem

Modern data analytics platforms ingest, process, and report on continuously growing volumes of transactional and operational data — sales records, sensor logs, clickstreams, and financial entries. A large share of reporting features (top-N dashboards, ranked leaderboards, percentile calculations, trend charts, and export-ready tables) depend on the data being sorted along one or more fields such as revenue, timestamp, or user ID. As dataset sizes scale from thousands to millions of rows, the efficiency of the sorting stage becomes a direct bottleneck on report-generation latency, memory footprint, and overall system responsiveness. Choosing an inappropriate sorting strategy can slow down dashboards, increase server memory pressure, or cause inconsistent ordering of tied records in ways that confuse end users.

2. Problem Statement

Given a large, unsorted dataset of analytical records (for example, daily revenue values across thousands of transactions), design and justify a sorting strategy for the platform's reporting engine that offers predictable performance, acceptable memory usage, and correctness guarantees (including stability where required), and compare Merge Sort and Quick Sort — both divide-and-conquer algorithms — to determine which is best suited to this workload under varying data sizes and distributions.

3. Algorithm Used

Two divide-and-conquer sorting algorithms are analyzed and implemented for this case study:

- **Merge Sort:** Repeatedly splits the array into two halves, recursively sorts each half, and merges the sorted halves back together in linear time.

- **Quick Sort:** Selects a pivot element, partitions the array into sub-arrays of smaller and larger elements relative to the pivot, and recursively sorts the partitions in place.

Both algorithms follow the classic divide-and-conquer pattern of (1) dividing the problem into smaller sub-problems, (2) conquering each sub-problem recursively, and (3) combining the sub-results into a final solution — but they differ sharply in how the "combine" step is performed and how much extra memory that step requires.

4. Justification for Choosing the Algorithm (Why This Algorithm?)

For the analytics platform's core reporting pipeline, Merge Sort is recommended as the default sorting algorithm, for the following reasons:

- **Predictable performance:** Merge Sort guarantees $O(n \log n)$ time in the best, average, and worst cases, so report-generation latency stays predictable even on adversarial or already-sorted input — a real risk with periodic or pre-aggregated analytics data.
- **Stability:** Merge Sort preserves the relative order of records with equal keys, which is essential when a secondary report re-sorts data that was already ordered by timestamp or ID.
- **Scalability for big data:** Merge Sort's linear merge step and easily divisible sub-arrays make it naturally suited to parallel and distributed processing (e.g., MapReduce-style external sorts), which analytics platforms rely on for very large datasets that exceed memory.

However, Quick Sort remains the better choice for smaller, in-memory batches (typical of ad-hoc, interactive dashboard queries) because its low constant-factor overhead and in-place partitioning typically make it faster in practice and lighter on memory when worst-case pivot behaviour is avoided (e.g., via randomized or median-of-three pivot selection). A hybrid strategy — Quick Sort for small/medium in-memory queries with a safeguard against worst-case input, and Merge Sort for large, disk-backed, or stability-sensitive batch reports — gives the platform the best of both algorithms.

5. Complexity Analysis

Metric	Merge Sort	Quick Sort	Notes
Best Case	$O(n \log n)$	$O(n \log n)$	Balanced splits
Average Case	$O(n \log n)$	$O(n \log n)$	Typical workloads
Worst Case	$O(n \log n)$	$O(n^2)$	Quick Sort degrades on poor pivots
Space Complexity	$O(n)$	$O(\log n)$	Merge needs auxiliary arrays
Stability	Stable	Not stable	Affects tie-breaking on equal keys
In-place?	No	Yes	Memory usage implication

Merge Sort's recursion divides the array into two halves at every level ($\log n$ levels), and the merge operation touches every element once per level, giving the recurrence $T(n) = 2T(n/2) + O(n)$, which resolves to $O(n \log n)$ in all cases. Quick Sort has the same recurrence only when the pivot splits the array evenly; a consistently poor pivot choice (e.g., always picking the first element on already-sorted data) produces $T(n) = T(n-1) + O(n)$, which resolves to $O(n^2)$.

6. Manual Execution (Step-by-Step Process)

6.1 Merge Sort — trace on [82, 15, 63, 4]

- Split [82, 15, 63, 4] into [82, 15] and [63, 4].

- Split [82, 15] into [82] and [15]; split [63, 4] into [63] and [4]. Single elements are already sorted.
- Merge [82] and [15] $\rightarrow$ [15, 82]. Merge [63] and [4] $\rightarrow$ [4, 63].
- Merge [15, 82] and [4, 63] by comparing fronts: $4 < 15 \rightarrow$ take 4; $15 < 63 \rightarrow$ take 15; $82 > 63 \rightarrow$ take 63; take 82 $\rightarrow$ result [4, 15, 63, 82].

6.2 Quick Sort — trace on [82, 15, 63, 4]

- Choose pivot = 63 (middle element).
- Partition: elements $< 63 \rightarrow$ [82? no, 15, 4]; elements $= 63 \rightarrow$ [63]; elements $> 63 \rightarrow$ [82].
- Recursively sort [15, 4]: pivot = 4 (or 15); partitions give [4] and [15] $\rightarrow$ combine $\rightarrow$ [4, 15].
- Combine: [4, 15] + [63] + [82] $\rightarrow$ result [4, 15, 63, 82].

Both traces converge on the identical sorted output [4, 15, 63, 82], confirming correctness; the difference lies in how the recursive work is organised — Merge Sort's cost is concentrated in the merge/combine phase, while Quick Sort's cost is concentrated in the partition/divide phase.

7. Applications

- Sorting ranked reports (top customers by revenue, top products by sales volume).
- Preparing time-series data for trend charts and moving-average calculations.
- External/distributed sorting of very large log or transaction files that exceed available RAM (Merge Sort).
- Fast in-memory sorting for interactive, ad-hoc query results in dashboards (Quick Sort).
- Database index construction and query optimizer sort operators.

8. Advantages

8.1 Merge Sort

- Guaranteed $O(n \log n)$ performance regardless of input order.
- Stable — preserves order of equal-key records, important for multi-column reports.
- Well suited to external sorting and parallel/distributed processing of huge datasets.

8.2 Quick Sort

- Low constant factors and in-place partitioning typically make it faster in practice for in-memory data.
- Space-efficient — $O(\log n)$ auxiliary space versus Merge Sort's $O(n)$.
- Cache-friendly due to in-place operation on contiguous memory.

9. Limitations

9.1 Merge Sort

- Requires $O(n)$ additional memory for the merge step, which is costly for very large in-memory datasets.
- Not in-place, so it is generally slower in practice than a well-tuned Quick Sort on typical in-memory workloads.

9.2 Quick Sort

- Worst-case $O(n^2)$ performance on poorly chosen pivots or adversarial/sorted input.
- Not stable by default, which can break secondary sort ordering in reports.
- Deep recursion on unbalanced partitions can risk stack overflow without safeguards such as tail-call optimisation or switching to insertion sort on small partitions.

10. Comparison with Other Algorithms

Algorithm	Avg. Time	Worst Time	Space	Stable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

11. Screenshot of the Program

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5  typedef struct {
6      int transaction_id;
7      int user_id;
8      double amount;
9  } Record;
10 Record* create_dataset(int size) {
11     Record* data = (Record*)malloc(size * sizeof(Record));
12     for (int i = 0; i < size; i++) {
13         data[i].transaction_id = 1001 + i;
14         data[i].user_id = 200 + i;
15         data[i].amount = (i % 2 == 0) ? 150.50 : 50.25;
16     }
17     return data;
18 }
19 void print_dataset(Record* data, int size) {
20     for (int i = 0; i < size; i++) {
21         printf("ID: %d | User: %d | Amount: $%.2f\n",
22             data[i].transaction_id, data[i].user_id, data[i].amount);
23     }
24     printf("\n");
25 }
26 void merge(Record* arr, int left, int mid, int right) {
27     int n1 = mid - left + 1;
28     int n2 = right - mid;
29     Record* L = (Record*)malloc(n1 * sizeof(Record));
30     Record* R = (Record*)malloc(n2 * sizeof(Record));
31     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
32     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
33     int i = 0, j = 0, k = left;
34     while (i < n1 && j < n2) {
35         if (L[i].amount <= R[j].amount) {
36             arr[k] = L[i];
37             i++;
```

```

38 ~         } else {
39 ~             arr[k] = R[j];
40 ~             j++;
41 ~         }
42 ~         k++;
43 ~     }
44 ~     while (i < n1) arr[k++] = L[i++];
45 ~     while (j < n2) arr[k++] = R[j++];
46 ~     free(L);
47 ~     free(R);
48 ~ }
49 ~ void merge_sort(Record* arr, int left, int right) {
50 ~     if (left < right) {
51 ~         int mid = left + (right - left) / 2;
52 ~         merge_sort(arr, mid + 1, right);
53 ~         merge_sort(arr, left, mid);
54 ~         merge(arr, left, mid, right);
55 ~     }
56 ~ }
57 ~ void swap(Record* a, Record* b) {
58 ~     Record temp = *a;
59 ~     *a = *b;
60 ~     *b = temp;
61 ~ }
62 ~ int partition(Record* arr, int low, int high) {
63 ~     int random_pivot = low + rand() % (high - low + 1);
64 ~     swap(&arr[random_pivot], &arr[high]);
65 ~     double pivot = arr[high].amount;
66 ~     int i = (low - 1);
67 ~     for (int j = low; j <= high - 1; j++) {
68 ~         if (arr[j].amount < pivot) {
69 ~             i++;
70 ~             swap(&arr[i], &arr[j]);
71 ~         }
72 ~     }
73 ~     swap(&arr[i + 1], &arr[high]);
74 ~     return (i + 1);

```

```

75 ~ }
76 ~ void quick_sort(Record* arr, int low, int high) {
77 ~     if (low < high) {
78 ~         int pi = partition(arr, low, high);
79 ~         quick_sort(arr, low, pi - 1);
80 ~         quick_sort(arr, pi + 1, high);
81 ~     }
82 ~ }
83 ~ int main() {
84 ~     srand(time(NULL));
85 ~     int size = 6;
86 ~     printf("=== INITIAL ANALYTICS DATASET ===\n");
87 ~     Record* merge_data = create_dataset(size);
88 ~     Record* quick_data = (Record*)malloc(size * sizeof(Record));
89 ~     memcpy(quick_data, merge_data, size * sizeof(Record));
90 ~     print_dataset(merge_data, size);
91 ~     printf("=== PROCESSING MERGE SORT (Guaranteed Stable) ===\n");
92 ~     merge_sort(merge_data, 0, size - 1);
93 ~     print_dataset(merge_data, size);
94 ~     printf("=== PROCESSING QUICK SORT (Unstable In-Place Partition) ===\n");
95 ~     quick_sort(quick_data, 0, size - 1);
96 ~     print_dataset(quick_data, size);
97 ~     free(merge_data);
98 ~     free(quick_data);
99 ~     return 0;
100 ~ }

```


12. Output Screenshot

```
==== INITIAL ANALYTICS DATASET ====
ID: 1001 | User: 200 | Amount: $150.50
ID: 1002 | User: 201 | Amount: $50.25
ID: 1003 | User: 202 | Amount: $150.50
ID: 1004 | User: 203 | Amount: $50.25
ID: 1005 | User: 204 | Amount: $150.50
ID: 1006 | User: 205 | Amount: $50.25

==== PROCESSING MERGE SORT (Guaranteed Stable) ====
ID: 1002 | User: 201 | Amount: $50.25
ID: 1004 | User: 203 | Amount: $50.25
ID: 1006 | User: 205 | Amount: $50.25
ID: 1001 | User: 200 | Amount: $150.50
ID: 1003 | User: 202 | Amount: $150.50
ID: 1005 | User: 204 | Amount: $150.50

==== PROCESSING QUICK SORT (Unstable In-Place Partition) ====
ID: 1004 | User: 203 | Amount: $50.25
ID: 1002 | User: 201 | Amount: $50.25
ID: 1006 | User: 205 | Amount: $50.25
ID: 1001 | User: 200 | Amount: $150.50
ID: 1003 | User: 202 | Amount: $150.50
ID: 1005 | User: 204 | Amount: $150.50

...Program finished with exit code 0
Press ENTER to exit console.
```

13. Conclusion

Both Merge Sort and Quick Sort are efficient divide-and-conquer algorithms well suited to the general problem of sorting large analytical datasets, but they optimize for different priorities. Quick Sort is preferable for fast, in-memory, interactive dashboard queries where average-case speed and low memory overhead matter most and a randomized pivot can guard against worst-case behaviour. Merge Sort is preferable for large-scale batch reporting, external/distributed sorting, and any scenario requiring stability or a guaranteed performance ceiling. For a production data analytics platform that must serve both interactive queries and large batch reports reliably, a hybrid approach — Quick Sort (with randomized pivots and an insertion-sort fallback for small partitions) for in-memory interactive workloads, and Merge Sort for large, stability-sensitive, or externally-sorted batch pipelines — offers the best overall balance of speed, memory efficiency, and correctness.